



Methodology for RTL DFT Violation and Coverage Analysis

Justin Mei, Ken Wang

Beijing Zhaoxin Electronic Technology Co., Ltd
Haidian District, Beijing, China

JustinMei@zhaoxin.com

KenWang@zhaoxin.com

ABSTRACT

Traditionally, DFT analysis and coverage analysis are done after scan synthesis. It is expensive iteration of RTL modification, scan synthesis and DFT simulation, if DFT issue is found. In addition to that, the time to market is shorter and shorter. It is urgent to develop a methodology to do DFT violation and coverage analysis in design early phase. This paper proposals the methodology about how to cover DFT issue and analyze coverage in RTL. Moreover, it is applied to real project and result is shared in the finial.

Table of Contents

Methodology for RTL DFT Violation and Coverage Analysis	1
1. Introduction.....	3
2. Project Environment Setup	3
3. Stuck ATPG test coverage analysis.....	6
3.1. Async_07 violations.....	7
3.2. Clock_11 violations.....	8
4. Speed ATPG test coverage analysis.....	9
5. Conclusions.....	12
6. References.....	12

Table of Figures

Figure 1. Traditional atpg test coverage flow	3
Figure 2. New atpg test coverage flow	3
Figure 3 dft/dft_best_practice goal check report.....	6
Figure 4. Stuck_at_coverage_auditrpt.....	7
Figure 5. Async_07 report.....	7
Figure 6. Async_07 violation DFF.....	8
Figure 7. Async_07 violation DFF debug.....	8
Figure 8. Clock_11 violation DFF debug.....	9
Figure 9. Test Coverage a fter clearing the Async_07 and Clock_11 violation.....	9
Figure 10. dft/dft_dsm_best_practice goal check report.....	10
Figure 11. Atspeed_11 violation DFF debug.....	11
Figure 12 Test Coverage a fter clearing the Atspeed_11 violation.....	11

Table of Tables

Table 1. SpyGlass & TMAX Test Coverage Compare	12
--	----

1. Introduction

SoC is very complex system require multiple partitions or Intellectual Properties (IPs) between sub-teams involved in its development. Handoff quality issues waste a lot of time to been deployed on many designs before they resulted in downstream design re-work. For example, DFT team found one clock in the design was'nt controlled in ATPG mode, and the design's test coverage was a little low. Therebefor, they release this issue to RD, RD team need to re-coding rtl design and compile the rtl design to gate netlist, then add scan chain, mbist. At last, DFT team check design's test coverage again. This re-work may be iterated several times. If we can do dft violation analysis in RTL level, this situation will be reduced and it has a great effective process. This paper introduct how to use SpyGlass tool to check DFT violation and how to impreove test coverage.

Test coverage mainly include stuck atpg test coverage and speed atpg test coverage(transition test coverage). The traditional test coverage report can be got by Figure 1 flow. The spyglas test coverage report flow shown as Figure 2. The stuck atpg test coverage checked by SpyGlass's dft/dft_best_practice goal, speed atpg test coverage checked by SpyGlass's dft/dft_dsm_best_practice goal.

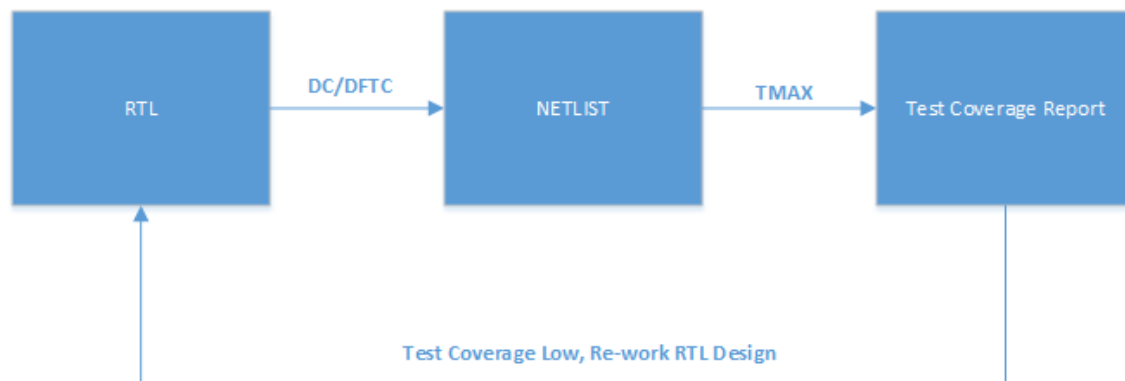


Figure 1. Traditional atpg test coverage flow

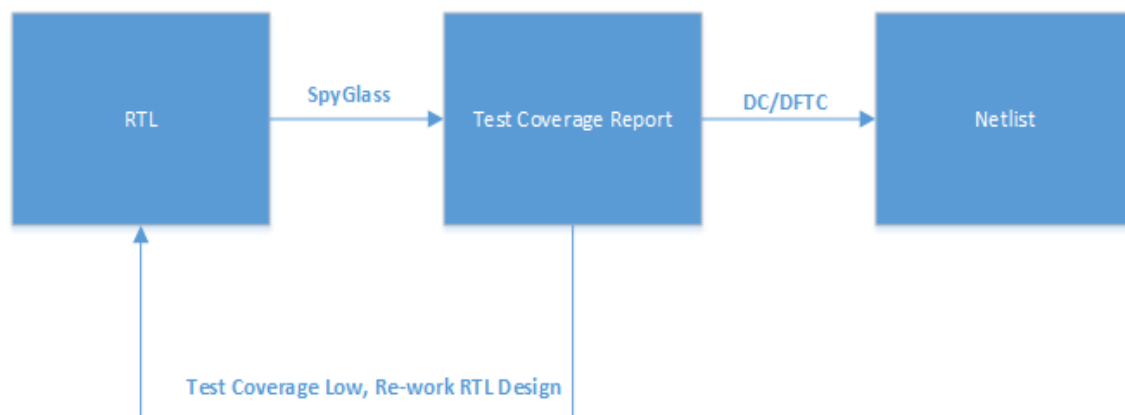


Figure 2. New atpg test coverage flow

2. Project Environment Setup

Spyglass has two different ways to run project flow. Using Tool Command Language (Tcl) interface:

```
sg_shell -64bit -tcl $(SG_TCL) -shell_log_file sg_shell.log
```

or Using Console Interface:

```
spyglass -run -project $(SG_TCL) -batch -goal "dft/dft_best_practice,
dft/dft_dsm_best_practice, dft/dft_dsm_best_practice"
```

Console is used to solve various design issues in the early stages of the design development process. If the design database is huge, we perform to use tcl interface. The Tcl interface enables you to interactively control the flow of various stages during a design analysis session. Use this interface to perform a variety of operations, such as compiling a design, applying constraints, configuring tool settings, controlling flow. Below is our spyglass.tcl:

```
set sg_run_time [benchmark -start sg_begin]
set TOP $env(TOP)
set ATTRIBUTE $env(ATTRIBUTE)
set SG_LIB $env(SG_LIB)
set SG_VMD $env(SG_VMD)
set SG_DESIGN $env(SG_DESIGN)
set STOP_MOD $env(STOP_MOD)
set DONT_SCAN $env(DONT_SCAN)
set SCAN_WRAP $env(SCAN_WRAP)
set SCAN_CONFIG $env(SCAN_CONFIG)
set GOAL $env(GOAL)
set EXIT $env(EXIT)
set DIR $env(DIR)

## define project dir
set project /${TOP}.project.prj
new_project $project -force
#add timing lib
source $SG_LIB
#add verilog lib
read_file -type sourcelist $SG_VMD
#add design
source $SG_DESIGN
set_option top $PART
## Explicitly Black box some of the modules in ${DESIGN_TOP}
source $STOP_MOD
## Explicitly specify modules/instances that are not to be ATPG tested
read_file -type sgdc $DONT_SCAN
```

```
## Explicitly scan_wrap black boxes in the design that are have a  
bypass register collar
```

```
read_file -type sgdc $SCAN_WRAP
```

```
## Read in constraints to define test modes
```

```
read_file -type sgdc $SCAN_CONFIG
```

```
set_option nosavepolicy { all }
```

```
set_option savepolicy { dft dft_dsm }
```

```
set_option enable_save_restore yes
```

```
set_option allow_module_override yes
```

```
link_design -top $PART
```

```
current_methodology
```

```
$env(SPYGLASS_HOME)/GuideWare/2016.06/block/rtl_handoff
```

```
if { [string match "*STCOV*" "$GOAL"] } {
```

```
current_goal dft/dft_best_practice
```

```
run_goal
```

```
}
```

```
if { [string match "*SPCOV*" "$GOAL"] } {
```

```
current_goal dft/dft_dsm_best_practice
```

```
run_goal
```

```
}
```

```
save_project
```

```
benchmark -show $sg_run_time -h
```

```
if { $EXIT == "yes" } {
```

```
close_project
```

```
exit
```

```
}
```

\$TOP is the partition module name. \$SG_LIB is timing tcl, the timing lib can be read by follow command:

```
read_file -type gateslib stdcell.lib
```

\$SG_DESIGN is read design file:

```
read_file -type Verilog TOP_module.v
```

\$SG_VMD is the verilog library file, or use below command to specify:

```
set_option v Verilog_module.v
```

\$STOP_MOD list all the don't check module. \$DONT_SCAN list all the don't scan module.

\$SCAN_WRAP list the sram bypass module. \$SCAN_CONFIG define all the dft signal values in test mode. \$ATTRIBUTE can be setted to "STCOV_SPCOV", then the tool will analysis the stuck atpg coverage and speed atpg coverage. After this script completed, use below command to enter console gui do debug and analysis.

```
sg_shell> gui_start
```

3. Stuck ATPG test coverage analysis

Figure 3 list the vast majority of the dft_best_practice rules. Right mouse click on coverage_audit in the message tree and select report. A coverage audit profile of the design determine the types of design issues that are impacting testability as Figure 4 shown. This report categorizes design issues that impact ATPG Test Coverage into 9 categories, prioritizing the order of issues that will most likely have the biggest impact on ATPG coverage loss. Looking at issue #2, "Flip-flops made scannable", one can see that if all F/F's are made scannable, then the test coverage will rise from 0.2% to 98.9%, a 98.7% coverage improvement. The column on the right provides the guidance on which rule violations should be reviewed and fixed; for scannability this are the Async_07 and Clock_11 violations.



Figure 3 dft/dft_best_practice goal check report

For Top Module 'VPIL'

Base stuck-at fault coverage :0.2
Base stuck-at test coverage :0.2

Expected stuck-at coverage data after each step.

	Expected stuck-at coverage		
	FC	TC	Action
0. Original Design	0.2	0.2	No DFT changes are made. It assumes 'force_scan' declared flip-flops scannable. Use Info_forcedScan to detect such flip-flops
1. PIs and POs made controllable & observable	0.2	0.2	set dft_treat_primary_inputs_as_x_source & dft_treat_primary_outputs_as_unobservable to off (No action required)
2. Flip-flops made scannable	98.9	98.9	Async_07 & Clock_11
3. Scan-wrap black boxes	99.0	99.0	Info_scanwrap
4. Latches made Transparent	99.0	99.0	Latch_08 & Info_testmode(capture) (No action required)
5. Combinational Loops made controllable	99.0	99.0	Topology_01 & non-X value loops
6. Testmode/Tied pins made controllable	99.3	99.3	Info_synthRedundant, Info_untestable and Info_pwrGndSim. Add -scanshift switch to test_mode constraint, if appropriate
7. Hanging nets made controllable	99.3	99.4	TA_09
8. Tristate enables made observable	99.3	99.4	TA_09 (No action required)
9. 'force_ta' and 'test_point' constraint pins made testable	99.3	99.4	Make pins or nets having 'force_ta' or 'test_point' constraint testable (No action required)
10. 'no_scan' flip-flops made scannable	100.0	100.0	Use Info_noScan & Info_inferredNoScan to view all no_scan flip-flops

Figure 4. Stuck_at_coverage_auditrpt

3.1. Async_07 violations

The Async_07 rule reports violation for flip-flops where the asynchronous set/reset source is active during the shift mode. The rule reports asynchronous source and the number of flip-flops, fed by this source, that are not inactive. This rule verifies that all the sources of asynchronous sets and resets are disabled in the shift mode.

Double-click on the violation message. SpyGlass has done root cause analysis by identifying groups of F/F's that have a common fan-in control logic to the F/F clock pins. In the first violation, there is a group of 50134 F/F's. The second violation is a group of 6008 F/F's. As Figure 5 shown.

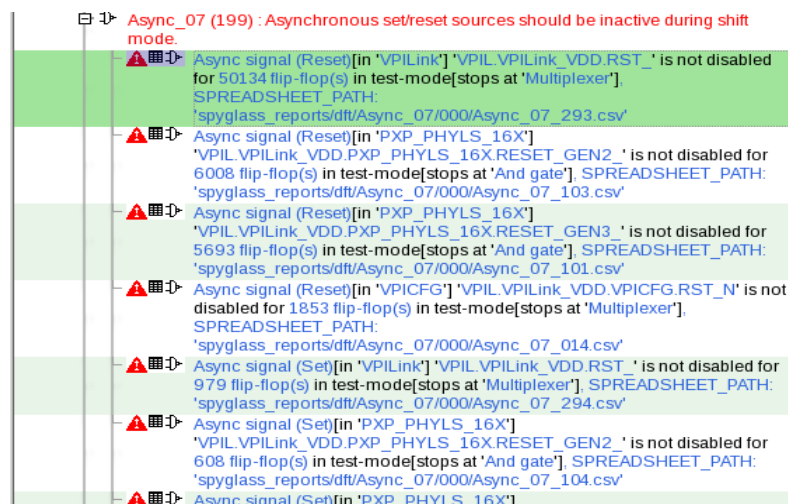


Figure 5. Async_07 report

Double-click on the first violation “Async signal (Reset)[in 'VPILink'] 'VPIL.VPILink_VDD.RST_' is not disabled for 50134 flip-flop(s) in test-mode” You will see a spreadsheet listing the 1524 F/F's as Figure 6 shown. They are grouped as having a common root cause with the reset not being defined or a defined reset path in the fan-in logic is blocked.

C1	B Instance Name
1	VPIL.VPILink_VDD.CFGCLKEN_ARBQ2_reg
2	VPIL.VPILink_VDD.CFGCLKEN_ARBQ1_reg
3	VPIL.VPILink_VDD.MSI_ADS_GQ2_reg
4	VPIL.VPILink_VDD.MSI_ADS_GQ1_reg
5	VPIL.VPILink_VDD.TVRRDY_pending_reg
6	VPIL.VPILink_VDD.TVRRDY_reg
7	VPIL.VPILink_VDD.TVNPRID_0Q_reg
8	VPIL.VPILink_VDD.TVNPRID_1Q_reg
9	VPIL.VPILink_VDD.TVNPRID_2Q_reg
10	VPIL.VPILink_VDD.TVWDQ_reg[63]
11	VPIL.VPILink_VDD.TVWDQ_reg[62]

Figure 6. Async_07 violation DFF

Click on one of the F/F hyperlinks in the spreadsheet and click on Schematic (IS) or type the letter “i” (hot key) to open the incremental schematic. The MUX gate is shown as blocking the propagation of a constrained reset, Trace the MUS’s pin and you can find the “TEST_ATPGEN” port has’nt test mode value as Figure 7 shown. Use the below command to add test mode value. “-scanshift -capture -captureATspeed” means “TEST_ATPGEN” port is 1 logic when the design is in scanshift mode , stuck atpg capture mode or speed atpg capture mode.

```
test_mode -name "TEST_ATPGEN" -value 1 -scanshift -capture
-captureATspeed
```

The others Async_07 violation have the same debug ways.

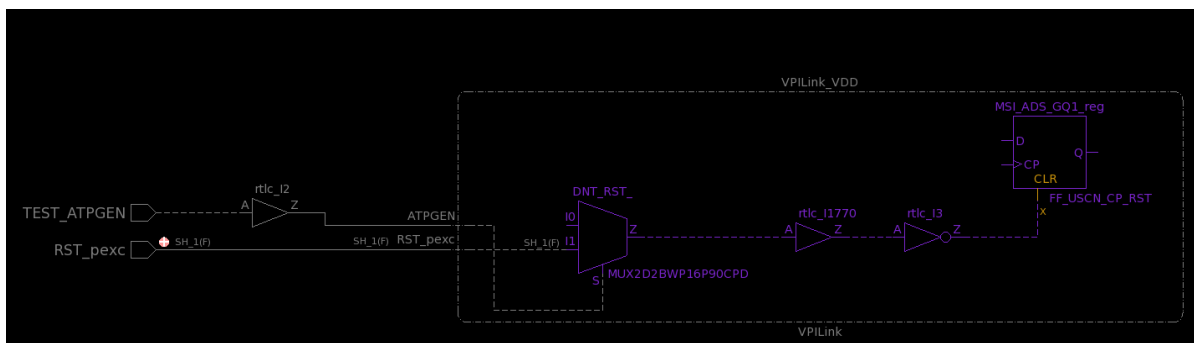


Figure 7. Async_07 violation DFF debug

3.2. Clock_11 violations

The Clock_11 rule reports violation for internally generated clocks that are not testclock-controlled in the shift mode. Mux-scan design requires that scan flip-flops be test clock controlled during scan

shift. The Clock_11 rule verifies that each clock source receives a pulse from a test clock pin in scan shift. Like Async_07 violation's debug way, We can find one hadn't defined. Use below command to add shift clock. "-testclock" means to add this port as shift clock.

```
clock -name "FVCLK" -domain "FVCLK_dom" -testclock
```

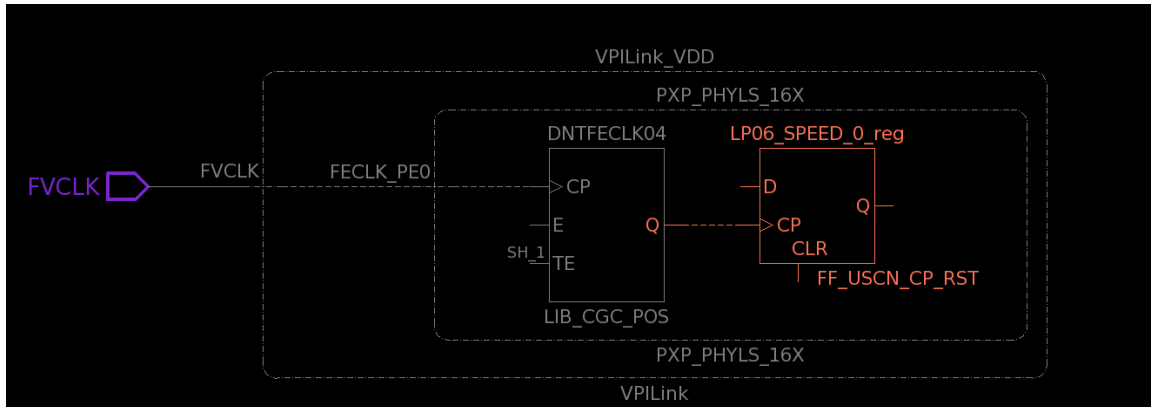


Figure 8. Clock_11 violation DFF debug

Clear all the Async_07 and Clock_11 violation, re-run the project flow. The test coverage improve to 98.9%. As Figure 9 shown.

Expected stuck-at coverage			
	FC	TC	Action
0. Original Design	98.9	98.9	No DFT changes are made. It assumes 'force_scan' declared flip-flops scannable. Use Info_forcedScan to detect such flip-flops
1. PIs and POs made controllable & observable	98.9	98.9	set dft_treat_primary_inputs_as_x_source & dft_treat_primary_outputs_as_unobservable to off (No action required)
2. Flip-flops made scannable	98.9	98.9	Async_07 & Clock_11 (No action required)
3. Scan-wrap black boxes	99.0	99.0	Info_scanwrap
4. Latches made Transparent	99.0	99.0	Latch_08 & Info_testmode(capture) (No action required)
5. Combinational Loops made controllable	99.0	99.0	Topology_01 & non-X value loops (No action required)
6. Testmode/Tied pins made controllable	99.3	99.3	Info_synthRedundant, Info_untestable and Info_pwrGndSim. Add -scanshift switch to test_mode constraint, if appropriate
7. Hanging nets made controllable	99.4	99.4	TA_09
8. Tristate enables made observable	99.4	99.4	TA_09 (No action required)
9. 'force_ta' and 'test_point' constraint pins made testable	99.4	99.4	Make pins or nets having 'force_ta' or 'test_point' constraint testable (No action required)
10. 'no_scan' flip-flops made scannable	100.0	100.0	Use Info_noScan & Info_inferredNoScan to view all no_scan flip-flops

Figure 9. Test Coverage after clearing the Async_07 and Clock_11 violation

4. Speed ATPG test coverage analysis

The most obvious difference between stuck atpg sgdc and speed atpg sgdc is speed clock. The first step to run dft/dft_dsm_best_practice goal is to explicitly identify which clock sources will have at-speed clocks. "-atspeed" option to the sgdc clock constraint specifies clocks to be considered as at-

speed clock sources. We can assumed the previously defined shift clock which come from top occ as speed clock, just add “-atspeed” option:

```
clock -name FVCLK -domain VICLK_dom -testclock -atspeed
```



Figure 10. dft/dft_dsm_best_practice goal check report

After this goal completed, Speed atpg test coverage is 97.0% which has been reported as Figure 10 shown. Atspeed_11 rule reports violation for clock domains that are not controlled by an atspeed clock in the test mode. Clock logic should be designed such that each flip-flop in the design is driven by an at-speed clock in the test mode. Clear this error to meet the high test coverage requirements..

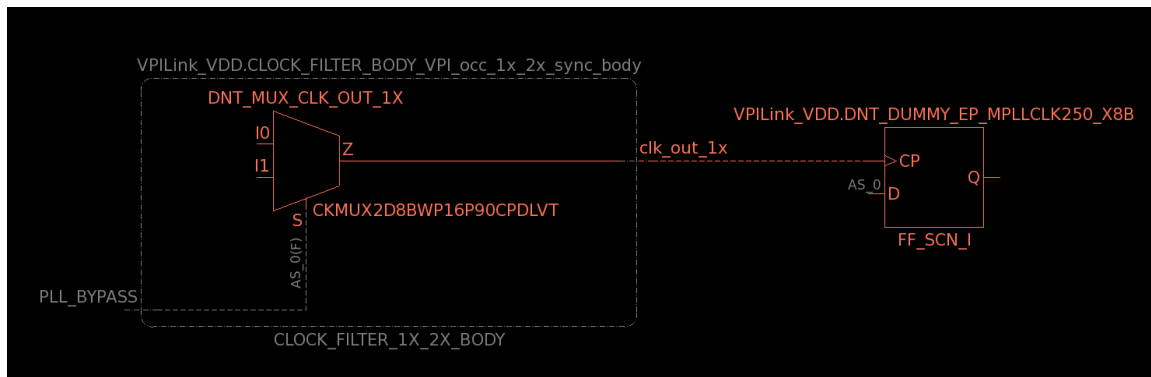


Figure 11. Atspeed_11 violation DFF debug

Trace the first violation DFF, we found the DFF's clock source come from OCC's output pin., as Figure 11 shown. Use below command to define "clk_out_1x" as speed clock.:

```
clock -name \
"VPILink_VDD.CLOCK_FILTER_BODY_VPI_occ_1x_2x_sync_body.DNT_MUX_CLK_OUT_1X.Z" -domain "VPI_occ_dom" -atspeed
```

<> For Top Module 'VPIL'

Atspeed launch method 'LAUNCH_ON_CAPTURE'

Base transition fault coverage :97.2
Base transition test coverage :97.2

Expected transition coverage data after each step.

	Expected transition coverage		
	Fault coverage	Test coverage	Action
1. Flip-flops made scannable	97.2	97.2	Clock_11 & Async_07 (No action required)
2. Black-box made scanwrapped	97.2	97.2	Info_scanwrap
3. 'no_scan' flip-flops made scannable	97.9	97.9	Info_noScan && Info_inferredNoScan
4. At speed test clocks	97.9	97.9	Atspeed_11 (No action required)
5. PI and PO used for launch and capture	97.9	97.9	set usePI & usePO to 'on' (No action required)
6. Controllable flip-flop d-pins	97.9	97.9	Atspeed_09 (No action required)
7. Clock domain crossing	99.1	99.1	Atspeed_06 & Diagnose_04
8. 'no_atspeed' faults removed	99.1	99.1	Atspeed_06 & Diagnose_04 (No action required)
9. Atspeed test mode	99.5	99.5	Info_testmode && Info_untestable
10. Tristate enables	99.5	99.5	TA_09 (No action required)
11. Undetected faults due to 'force_ta'	99.5	99.5	TA_09 (No action required)
12. Hanging terminals	99.5	99.5	TA_09
13. Flip-flops with q to d feedback	99.8	99.8	consider redesign
14. False path faults	99.8	99.8	Diagnose_03 (No action required)
15. Undetected faults due to PG nets	100.0	100.0	Info_pwrGndSim, Info_synthRedundant & Atspeed_09

Figure 12 Test Coverage after clearing the Atspeed_11 violation

Clear all the Atspeed_11 violation, re-run the project flow. The test coverage improve to 97.2% from 97.0% as Figure 12 shown. The report indicates the current coverage for the design. Follow each steps to improve the test coverage, such as : Info_scanwrap rule remind us which black-box need

to do bypass logic. The Atspeed_06 violations prompted us to try to avoid clock domain crossing and so on.

5. Conclusions

In order to compare the accuracy of SpyGlass's stuck atpg test coverage report, we compiled the same rtl design to netlist and do scan synthesis. After that, we run stuck atpg full coverage using TetraMAX. Table 1 is the result of the experiment. As can be seen from the table, SpyGlass's test coverage is almost equal to TetraMAX's test coverage. Average deviation is only 0.306%.

The test coverage result of SpyGlass tool prediction is credible. It can make us to modify the design to follow DFT rule in the RTL level, but not in netlist level. SpyGlass tool can greatly simplify our workflow and increase work efficiency.

Table 1. SpyGlass & TMAX Test Coverage Compare

Design	SpyGlass's Test Coverage Report	TetraMAX's Test Coverage Report	Deviation
VPIL_1	99.00%	98.86%	+0.14%
VPIL_2	98.90%	98.76%	+0.14%
SATA	98.90%	98.47%	+0.43%
USB3HC	99.70%	99.11%	+0.59%
PEXC	99.21%	98.98%	+0.23%

6. References

- [1] SpyGlass_TclShellInterface_UserGuide, Version 5.4.1, Synopsys.
- [2] SpyGlass_DFTRules_Reference, Version 5.4.1, Synopsys.
- [3] SpyGlass_DFTDSMRules_Reference, Version 5.4.1, Synopsys.
- [4] SpyGlass_DFTMethodology_GuideWare2.0_UserGuide, Version 5.4.1, Synopsys.